# TAPRIVO

Architecture & Production Deployment Guide

Sizing · Security · High Availability · Performance · Data Durability

Digital loyalty platform — Mobile (Flutter), Web (React PWA), API (Fastify)
Reference document · v1

# 1. System Overview

Taprivo is a digital loyalty platform that replaces paper punch cards. A customer joins a merchant's program and earns a **stamp** per visit; when the card reaches the merchant's threshold a **reward** (coupon) is issued and redeemed at the counter, resetting the card. Stamps and redemptions happen by **tapping the merchant's NFC tag** on the customer's phone, or via a short-lived **QR code**. All actions are validated server-side.

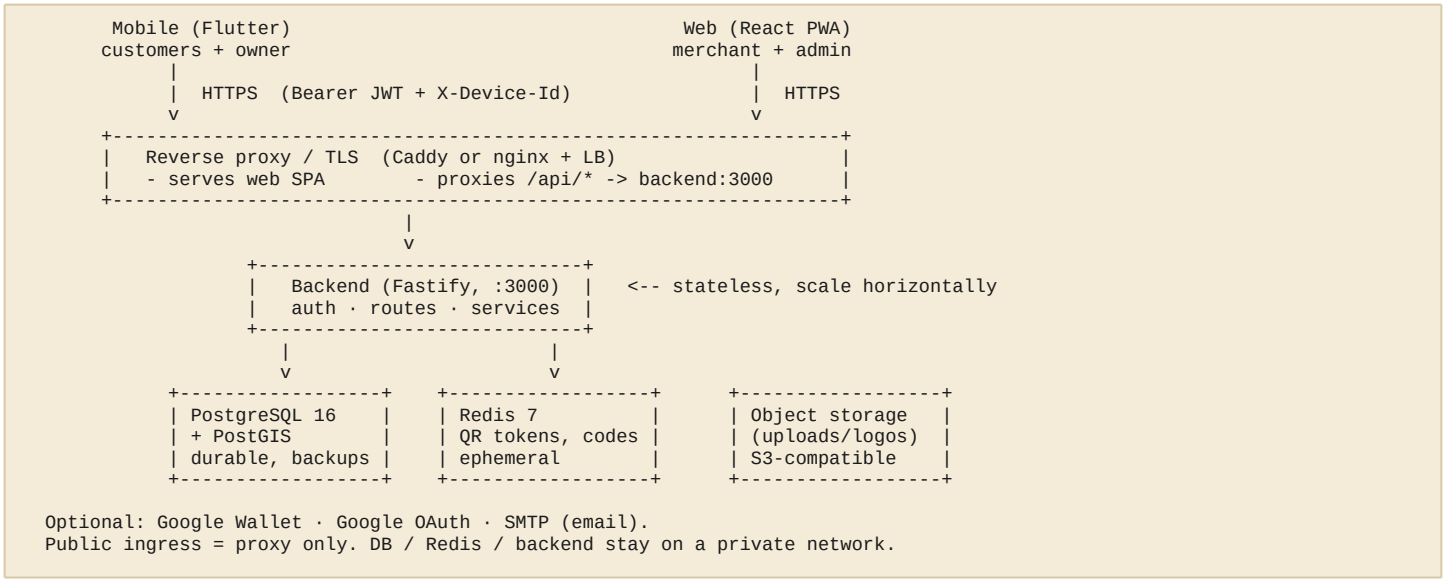
### Components

| Component | Tech | Audience / role |
|---|---|---|
| Mobile app | Flutter (Dart ≥ 3.6), Riverpod, Dio, nfc_manager, geolocator | Customers (cards, NFC tap, rewards, Google Wallet) + merchant scanning |
| Web app (PWA) | React 18, Vite 5, Tailwind v4, served by nginx | Merchant console + Admin console (+ client web) |
| API | Node 20, Fastify 4, TypeScript, Zod, JWT | All business logic, auth, validation |
| Database | PostgreSQL 16 + PostGIS | Durable data (users, cards, stamps, rewards, devices) |
| Cache | Redis 7 | Ephemeral: one-time QR tokens (~60 s), email/reset codes (~15 min) |

### Roles

| Role | Where | Capabilities |
|---|---|---|
| client | Mobile (+web) | Join programs, earn stamps (NFC/QR), view cards, redeem rewards, Google Wallet |
| merchant | Web + mobile owner | Configure program, provision NFC tags, scan QR, redeem rewards, stats/customers |
| admin | Web | Full CRUD on users & merchants, global stats & activity, all NFC devices |

## 1.1 Logical architecture




> **Key architectural fact for scaling:** the API is **stateless** — sessions live in Postgres (refresh tokens) and Redis (QR/codes). This means the backend can run as **N identical replicas** behind a load balancer. The stateful tier (Postgres, Redis, object storage) is where high-availability and durability investment goes.

## 1.2 Request & trust flow (NFC stamp)

1. Customer opens their card in the mobile app; the app arms the NFC reader.
2. Merchant taps their provisioned NFC tag on the phone → app reads the tag's hardware UID.
3. App calls `POST /nfc/validate` with the UID (and optionally GPS coords) over HTTPS, authenticated by a short-lived JWT.
4. Backend matches the UID to an active `nfc_devices` row, checks the merchant is NFC-enabled, optionally enforces the geofence, then increments the stamp and (if the card is full) issues a reward — all inside a DB transaction.

# 2. Deployment Tiers — "What to buy"

Pick the tier that matches your stage. You can start at Tier 0 and migrate up without code changes (the app is built to externalize state). Costs are approximate monthly ranges, EUR, illustrative — choose a region close to your users; the seed data is in Tunis, so an EU-South / North-Africa-adjacent region minimizes latency.

## Tier 0 — Pilot / MVP single VPS

Everything on one machine via Docker Compose (Postgres, Redis, backend, web, Caddy for TLS). Good for a demo, a single neighborhood, or up to a few thousand users. **Single point of failure** — acceptable only while piloting, and only with off-site daily backups.

| Item | What to buy | Spec | ~Cost/mo |
|---|---|---|---|
| App + data VPS | 1 × VPS (e.g. Hetzner CPX31 / equivalent) | 4 vCPU, 8 GB RAM, 80–160 GB SSD | €12–18 |
| Backups | S3-compatible object storage | daily pg_dump + uploads, off the VPS | €1–5 |
| Domain + TLS | 1 domain; certs free (Let's Encrypt via Caddy) | — | €1 |
| **Total** | | | ≈ **€15–25** |

RPO ≈ up to 24 h (last daily dump). RTO = time to rebuild a VPS + restore. No redundancy.

## Tier 1 — Production (recommended start) managed data + app VPS

Remove the database as a single point of failure and get point-in-time recovery. App stays on cheap VPS (stateless), data moves to managed services. This is the right baseline for a real public launch.

| Item | What to buy | Spec / why | ~Cost/mo |
|---|---|---|---|
| App nodes | 2 × small VPS (backend + web containers) | 2–4 vCPU, 4–8 GB each; 2 nodes = no app SPOF + rolling deploys | €20–40 |
| Load balancer | 1 × managed LB (or Caddy/HAProxy on a 3rd small node) | health-checked, TLS termination, spreads traffic | €5–15 |
| PostgreSQL | Managed Postgres w/ daily backup + **PITR** + standby | 2 vCPU, 4 GB, 50–100 GB; automated failover | €30–70 |

| | | | |
|---|---|---|---|
| Redis | Managed Redis (or co-located; data is ephemeral) | 1 GB | €10–20 |
| Object storage | S3-compatible bucket for `uploads/` | shared across app nodes; versioned | €1–5 |
| CDN | CDN in front of web + images | low latency static + cache | €0–20 |
| **Total** | | | ≈ **€75–170** |

RPO ≈ seconds–minutes (PITR/WAL). RTO ≈ minutes (managed failover). App survives one node loss.

## Tier 2 — High Availability / Scale cluster / multi-AZ

For tens of thousands+ of active users, strict SLAs, and no-SPOF at every tier. Kubernetes (managed) or a multi-node Docker/Nomad cluster across ≥ 2 availability zones.

| Item | What to buy | Spec / why | ~Cost/mo |
|---|---|---|---|
| Compute | Managed Kubernetes, 3+ nodes across 2–3 AZ | backend autoscaled (HPA); web on CDN | €150–400 |
| PostgreSQL | Managed HA Postgres, multi-AZ, sync standby + read replicas | automatic failover; replicas for read scaling | €150–500 |
| Redis | Managed Redis HA (Sentinel/cluster) | replicated, failover | €30–100 |
| LB / ingress | Cloud LB (multi-AZ) + WAF | DDoS/WAF protection, TLS | €20–60 |
| Storage + CDN | Multi-region object storage + global CDN | durability + global latency | €20–80 |
| Observability | Logs + metrics + alerting + uptime | Grafana/Loki or hosted | €20–80 |
| **Total** | | | ≈ **€400–1200+** |

RPO ≈ 0 (synchronous replication). RTO ≈ seconds–minute (auto-failover). Survives a full AZ outage.

> **Recommendation:** launch on **Tier 1**. It gives real durability (PITR) and removes the database SPOF for < €170/mo, and the stateless API lets you grow into Tier 2 by adding replicas — no rewrite. Tier 0 is only for a closed pilot.

# 3. Security

## 3.1 Already in the codebase

- **JWT access tokens** (~15 min) + **rotating refresh tokens** stored only as SHA-256 hashes; reuse of a revoked token revokes the whole session chain (theft protection).
- **Passwords** hashed with bcrypt (cost 10).
- **Optional device binding** (`ENFORCE_DEVICE_BINDING=true`) ties a session to `X-Device-Id` (anti account-sharing).
- **Input validation** with Zod on every route; **rate limiting** 120 req/min/IP.
- **Role guards**: `requireAuth` / `requireMerchant` / `requireAdmin`.
- **Per-merchant NFC secret**; one-time, HMAC-signed, 60 s QR tokens.
- **Server-side geofence** (PostGIS / Haversine) when coordinates are provided.

## 3.2 Must-do for production

| Area | Action |
|---|---|
| Transport | HTTPS everywhere (TLS 1.2+), HSTS at the proxy, redirect 80→443. Disable mobile cleartext (Android `usesCleartextTraffic`, iOS `NSAllowsLocalNetworking`). |
| Secrets | Fresh unique `JWT_SECRET`, `JWT_REFRESH_SECRET`, `QR_HMAC_SECRET`, `NFC_HMAC_SECRET` (`openssl rand -hex 32`); store in a secret manager, never in git. Strong DB password. |
| CORS | Set `CORS_ORIGIN` to exact web origin(s) — never `*` in prod. |
| Network | Only the proxy is public. Postgres/Redis/backend on a private network/VPC; DB firewalled to app nodes only. |
| Runtime | Run backend as non-root, compiled JS (not `tsx`), read-only FS where possible; drop the dev source bind-mount. |
| Edge | WAF + DDoS protection (Cloudflare or cloud LB). Consider a stricter auth rate-limit on `/auth/*`. |
| Supply chain | Pin/scan dependencies (npm audit, image scanning), automate patch updates, rebuild images regularly. |
| Data at rest | Encrypt DB volumes & backups; restrict and audit access. Personal data = emails, names, locations — handle per GDPR/local law (retention, deletion on request). |

| | |
|---|---|
| Mobile | Tokens already in Keychain/Keystore (`flutter_secure_storage`). Sign release builds with a safeguarded keystore; enable Play App Signing. |
| Operations | Audit logging (stamp/reward events already logged), least-privilege admin accounts, remove demo accounts before launch. |

# 4. High Availability (no single point of failure)

| Tier | HA strategy |
|---|---|
| Ingress / proxy | ≥ 2 LB instances or a managed multi-AZ LB; health checks remove unhealthy app nodes automatically. |
| Backend API | **Stateless** → **run** ≥ **2 replicas** across nodes/AZs. Rolling/blue-green deploys = zero-downtime releases. Liveness/readiness via `GET /health`. |
| Web (PWA) | Static assets on CDN (inherently redundant) + ≥ 2 nginx origins. Service worker keeps the app usable during brief blips. |
| PostgreSQL | Primary + synchronous standby with **automatic failover** (managed). Optional read replicas to offload heavy reads (stats/admin). |
| Redis | Replicated with Sentinel/cluster. Note: Redis holds only ephemeral data — a flush degrades gracefully (users re-generate QR / re-request codes), it does not lose business data. |
| Object storage | S3-class durability (11 nines) + versioning; multi-region for Tier 2. |

## Application notes that affect HA

- **Rate limit is in-memory per instance.** With multiple replicas the effective limit multiplies; for a strict global limit, back the limiter with Redis.
- **Uploads are a local volume today.** For multi-node/HA, switch merchant logo storage to the shared object storage (otherwise logos only exist on the node that received them).
- **DB pool is max 10 per instance.** Size `max_connections` and/or add PgBouncer so (replicas × 10) stays within the database's limit.

> **Target SLOs (Tier 1–2):** 99.9% API availability; automatic recovery from any single node/AZ failure with no operator action; zero-downtime deploys.

# 5. Performance & Response Time

| Lever | Action / status |
|---|---|
| Region proximity | Host close to users (EU-South / nearest region to Tunisia) to cut RTT — the single biggest latency factor for mobile. |
| CDN | Serve web bundle + merchant logos from a CDN; Vite hashes assets → 1-year immutable cache (already in nginx config). |
| Compression | gzip enabled in nginx for text/json/js/css. |
| DB indexing | Hot paths already indexed: `loyalty_cards(user_id)`, `(merchant_id)`, `stamp_events(merchant_id, scanned_at)`, `nfc_devices.uid` unique, etc. Add indexes as new query patterns appear; watch slow-query logs. |
| Caching | Use Redis to cache hot read-mostly data (e.g. public `/merchants` list) with short TTLs. Heavy analytics → read replica. |
| Connection pooling | Tune pg pool per replica; add PgBouncer at scale to avoid connection storms. |
| Payloads | Endpoints already return scoped lists (e.g. last 20 events, 50–60 feed items). Keep pagination as data grows. |
| Mobile UX latency | Optimistic UI + haptics on NFC tap make the perceived response instant; the network call confirms in the background. |
| Autoscaling | Tier 2: scale backend replicas on CPU/RTT (HPA). Because it's stateless, scaling is linear. |

> **Target:** p95 API response < 200 ms in-region for stamp/validate/list endpoints (simple indexed queries). The dominant cost is usually network RTT, addressed by region choice + CDN.

# 6. Data Durability — "no data loss"

Durability is about the **stateful tier**. The backend and web are disposable; what must never be lost is PostgreSQL (and merchant logos in object storage). Redis is ephemeral by design.

## 6.1 Layered protection

| Layer | Mechanism | Protects against |
|---|---|---|
| Replication | Synchronous standby (managed Postgres) | Primary node/disk failure → failover with RPO ≈ 0 |
| PITR / WAL | Continuous WAL archiving to object storage | Restore to any second; recover from bad deploy / accidental delete |
| Daily backups | Automated `pg_dump` / managed snapshots, retained 7–30 days | Logical corruption, ransomware; off-site copy |
| Off-site / cross-region | Copy backups to a second provider/region | Whole-region or provider outage |
| Object storage | Versioning + lifecycle on the uploads bucket | Overwritten/deleted logos |
| Restore drills | Periodically restore a backup to a scratch DB and verify | "Backups that don't restore" — the classic failure |

## 6.2 Application safeguards already present

- Stamp + reward writes run inside **DB transactions** (`tx()`) — no half-applied stamps.
- **Idempotent migrations** run on boot; `db/init.sql` sets the base schema on a fresh volume.
- Full **audit trail** in `stamp_events` (method, time, geo) and `rewards` (issued/redeemed) — business history is reconstructable.

## 6.3 Backup commands (Tier 0/1 self-managed)

```
# Daily dump (cron), then push to object storage
docker compose exec -T postgres pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" \
 | gzip > backup-$(date +%F).sql.gz
# Restore
gunzip -c backup-YYYY-MM-DD.sql.gz \
 | docker compose exec -T postgres psql -U "$POSTGRES_USER" "$POSTGRES_DB"
```

> **Targets:** Tier 1 → RPO ≤ 5 min (PITR), RTO ≤ 30 min. Tier 2 → RPO ≈ 0 (sync replication), RTO ≤ 5 min (auto-failover). Always keep ≥ 1 off-site copy.

# 7. App Distribution — iOS · Android · Web

## 7.1 Android

- Build a signed **App Bundle**: `flutter build appbundle --release --dart-define=API_BASE_URL=https://api.taprivo.com --dart-define=GOOGLE_SERVER_CLIENT_ID=<web-client-id>`.
- Create a release **keystore**, wire it in `android/app/build.gradle.kts`, store it safely; enable **Play App Signing**.
- Upload the `.aab` to **Google Play Console** (one-time $25 dev account). Register the release SHA-1/256 in the Google OAuth client for Sign-In.
- Remove `usesCleartextTraffic` for release (API is HTTPS).
- NFC: Android supports the hands-free continuous reader used by the app.

## 7.2 iOS

- Requires a **Mac** (or a CI like Codemagic/Bitrise) and an **Apple Developer Program** membership ($99/yr).
- Add the **Near Field Communication Tag Reading** capability + entitlement in Xcode; keep the existing NFC/location usage strings; remove `NSAllowsLocalNetworking`.
- Build & upload: `flutter build ipa --release --dart-define=API_BASE_URL=https://api.taprivo.com …` → Transporter/Xcode → **App Store Connect** → TestFlight → review.
- NFC on iOS is foreground-only → tap-to-scan UX (vs. Android hands-free).

## 7.3 Web (PWA)

- Built to static files (`vite build`) and served by the `web` nginx image, which also proxies `/api` to the backend. Put a CDN + TLS in front.
- Installable PWA with auto-updating service worker — no app store needed; instant updates on deploy.
- Build-time API origin: keep `VITE_API_URL=/api` with the bundled proxy, or point it at a dedicated API subdomain.

| Channel | One-time / recurring cost | Update mechanism |
|---|---|---|
| Google Play (Android) | $25 once | Upload new `.aab`; staged rollout |
| App Store (iOS) | $99 / year | Upload build; Apple review |
| Web PWA | hosting/CDN only | Instant on deploy (service worker) |

# 8. Reference — Environment & Sizing Cheat-Sheet

## Required backend secrets (generate with `openssl rand -hex 32`)

`JWT_SECRET` · `JWT_REFRESH_SECRET` · `QR_HMAC_SECRET` · `NFC_HMAC_SECRET` · plus `DATABASE_URL`, `REDIS_URL`.

## Production-critical env

`NODE_ENV=production` · `CORS_ORIGIN=https://app.taprivo.com` · `PUBLIC_BASE_URL=https://api.taprivo.com` · `ENFORCE_DEVICE_BINDING=true` (optional) · `UPLOAD_DIR` on a mounted/shared volume.

## Optional integrations

Google Wallet (`GOOGLE_WALLET_ISSUER_ID`, `GOOGLE_WALLET_SA_FILE`/`_JSON`) · Google OAuth (`GOOGLE_OAUTH_CLIENT_IDS`) · SMTP (`SMTP_HOST`, …). Unset = feature returns 503 / codes logged to console.

## Quick decision guide

| Your situation | Buy this tier |
|---|---|
| Demo / closed pilot, < few k users, cost-first | **Tier 0** — 1 VPS + off-site backups |
| Public launch, real users, durability required | **Tier 1** — 2 app VPS + LB + managed Postgres (PITR) + object storage + CDN |
| Scale, SLAs, no-SPOF, multi-AZ | **Tier 2** — managed K8s + HA Postgres/Redis + WAF + observability |

Costs are approximate and provider-dependent; validate with your chosen host. Full step-by-step commands (prod Dockerfile, compose override, Caddy TLS, keystore, smoke tests, backups) are in `DEPLOYMENT.md` at the repository root.